

# Ventanita

A WhatsApp order-taking bot for a small food business. No official API, no framework, no vendor.

Technical overview · v0.2.4 · What is shipped, and where it is going

---

## 1. What it is

---

Ventanita is a Python program that watches a WhatsApp Web browser window on a machine you own, reads it with OCR, and types replies back with real keystrokes. It is not an integration. It does not talk to Meta's Business API, does not register templates, and does not pay per message. It uses the same door a human uses: a linked browser session, a screen, and a keyboard.

```
unread badge → open chat → OCR → clean → look up customer → ask LLM → gate → type →  
close chat → loop
```

That is the whole architecture. Eight small modules, one job each, under `src/ventanita/`. The LLM is the only stochastic part; everything around it is if/then, so a wrong answer is traceable to one variable.

### Shipped through v0.2.4

- **Screen-driven detection.** The chat list is scanned for WhatsApp's unread-badge green. Badge position is not fixed — it shifts a couple of dozen pixels on the row showing the dropdown chevron — so the scan covers an x-band, not one column.
- **Drain loop for concurrent chats.** Every visible unread row is found, not just the topmost. The bot opens one, replies or skips, closes it, then rescans from scratch before touching the next, because sending a message reorders the list and invalidates every y-coordinate. Bounded per pass by `safety.max_chats_per_drain` (default 4).
- **Real per-chat multi-turn memory.** A messages table stores each customer's turns; they are replayed to the model as a proper messages array. Verified live on a follow-up question that only made sense with prior context.
- **Session-summary memory.** Replaying full history forever gets expensive and stale. After a quiet gap (`timing.session_gap_hours`, default 3), the turns since the last note are folded into a short written note (capped at 600 characters) by one cheap model call, and that note accumulates rather than being overwritten. New replies then see the current session's turns plus the note. No cron, no background thread — it is a check at the top of the existing per-chat path.
- **Typing-aware debounce.** If the chat row reads "typing" / "escribiendo" after the chat is opened, the bot waits (up to `timing.typing_wait_sec`, default 15) and re-reads, so it answers the finished thought instead of half a sentence.
- **Per-chat identity and dedup.** The OCR'd row label is the customer key, so histories no longer collapse into one record. Before spending an LLM call, the latest inbound line is compared against the last stored one; an unchanged re-read is closed without a reply.
- **Orders and payment status in SQLite.** Four tables in one file. `orders.status` has a fixed vocabulary (new → preparing → ready → paid, plus cancelled) enforced at the write, so typos cannot invent a fifth state. Orders record how they were paid; customers record a preferred method. Migrations are guarded `ALTER TABLE` statements on connect — additive only, no migration framework.

- **Chilango-Spanish persona.** Voice comes from the config-supplied system prompt, so per-brand is per-config file.
- **Safety rails.** F9 kill switch (X11 hooks, no root needed) aborts typing mid-message and refuses further sends until restart. A human gate holds orders above a configured peso threshold instead of confirming them. Active-hours sleep, randomized think delays, per-character typing jitter, and a chat allowlist. Failures log loudly rather than passing silently.

### What it runs on

| LAYER                | CHOICE                                                                                                                 |
|----------------------|------------------------------------------------------------------------------------------------------------------------|
| Language             | Python 3.11+                                                                                                           |
| Screen capture / OCR | mss + Tesseract via <code>pytesseract</code> (Spanish pack)                                                            |
| Input                | <code>pyautogui</code> for clicks, <code>xdotool</code> type for text ( <code>pyautogui</code> mangles accents on X11) |
| Storage              | <code>sqlite3</code> , <code>stdlib</code> , one file                                                                  |
| Model                | One HTTP call behind one function — swap the provider without touching anything else                                   |
| Config               | <code>config.yaml</code> plus <code>.env</code> for keys                                                               |

Setup is an afternoon: install Tesseract, `pip install -e .`, run the calibration helper once with the WhatsApp window pinned, start the loop. Running cost is cents a day in tokens. Everything — customers, orders, message history — lives in one file on the operator's disk.

### Known limits, stated plainly

- OCR never yields a phone number, so the display name is the identity key. Two contacts with the same name share a record; a misread name splits one. Documented, not solved.
- The window must stay pinned at a fixed position, size, and zoom. If it moves, recalibrate.
- The typing-wait is synchronous, so a burst of chats can queue behind each other. Accepted tradeoff for a single-window bot.
- Nothing in the OCR loop can observe that a payment happened. Payment fields exist and are written by hand or by an operator command — there is no gateway code.
- The 0.2.2–0.2.4 work came from a research pass between live sessions, not from watching it misbehave. Treat those defaults as first guesses until the next real run says otherwise.

## 2. Where it is going

### ROADMAP — NOT BUILT

Nothing in this section exists in the code today. These are directions, not commitments, and none of them has a date. The project's rule has been to build the atom first and leave the seams: a feature ships when a real kitchen has hit the edge case, not when it sounds good on a page. Read section 1 for what actually runs.

### Payment links that close the loop

Generate a Mercado Pago link for an order, attach it to that order row, and route the confirmation back to the right customer's chat when it settles. The naming is already in place — `mercado_pago` is one of the recorded payment methods today — so hand-entered rows keep the same label a real integration would use. The hard part is not the link. It is the return path: a webhook or poller has to identify which chat to notify, and the bot's

only handle on a customer is an OCR'd display name. That identity problem is upstream of the payment work.

### **Understanding voice notes and photos**

Customers send audio and pictures, not only typed text. Today OCR sees a bubble it cannot read, and the bot has nothing to say about it. The intended shape is transcription for audio and a small vision model on call for images — invoked only when the message is not plain text, feeding a description into the same prompt path that already exists, rather than becoming a second pipeline.

### **Spoken replies**

Answering a voice note with a voice note. Mechanically this is text-to-speech plus WhatsApp Web's attachment flow driven the same way typing is driven now. Open question is whether a synthetic voice is the right thing to put in front of a customer at all; that gets decided by trying it in the sandbox chat, not by arguing about it.

### **Local model or rented model, operator's choice**

The model already sits behind one function, which was the point. The direction is making a fully local model a first-class option — on-machine inference, no cloud dependency, no key, no per-token bill — while keeping pay-per-use against a cloud provider for operators who would rather rent quality than run hardware. Same code path, one config line. The brain is rented on purpose so it can stop being rented.

### **Supervision console**

A read-mostly view of chats, orders, and payment status, and the place a human marks an order paid or ready. This is the intended caller for the payment helpers that already exist in the database layer but have no trigger. Design notes only so far.

## **3. Why it is built this way**

---

- **Boring tech owns its failures.** OCR, SQLite, and mouse events have twenty years of known failure modes. When they break, you fix it yourself in four minutes. Cloud APIs break on someone else's schedule.
- **Freeze the world.** Pinned window, fixed resolution, dark mode, 100% zoom. Perception becomes data entry instead of computer vision. Make the environment dumber rather than the model smarter.
- **The AI is a part, not the product.** One replaceable component inside a deterministic box. The stochastic thing touches the reply text and nothing else.
- **Bugs get written down.** The changelog records what was found live and what was designed in the quiet, and says which is which. A claim about behavior is only trustworthy if you can see where it was verified.